

Remote Procedure Calls



COMS4113

COMS4113: Fundamentals of Distributed Systems

Instructor: Hubertus Franke

Columbia University

<https://frankeh.github.io/cu-ds1-2026-fall>

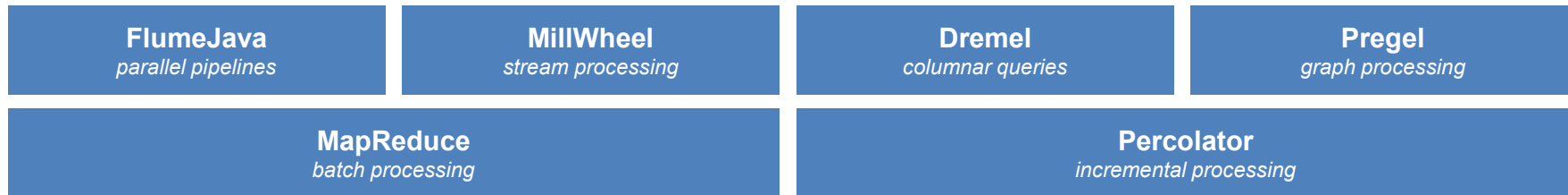
- Where RPC sits in the distributed-systems stack
- A short history: from RFC 707 to gRPC
- Motivation: why raw sockets are the wrong abstraction
- The RPC idea, and the stub/marshaling architecture
- Benefits — and the four places distribution leaks through
- Failure semantics: at-least-once, at-most-once, “exactly once”
- The critiques: Tanenbaum & van Renesse; Waldo et al.
- RPC in 2026: gRPC, Connect, HTTP/3, service meshes, kernel-bypass

- We have discussed the challenges of scalability, fault tolerance, semantics and performance in distributed systems.
- The general approach: build infrastructure that **raises the level of abstraction**, so applications do not re-solve these problems.
- Today we study the most basic such abstraction: **RPC**, the predominant communication mechanism in a distributed system.
- RPC is small enough to cover in one lecture, yet it already exhibits *most* of the challenges of the whole course.
- In particular, it is where you first meet the central fact of distribution: **components fail independently**.

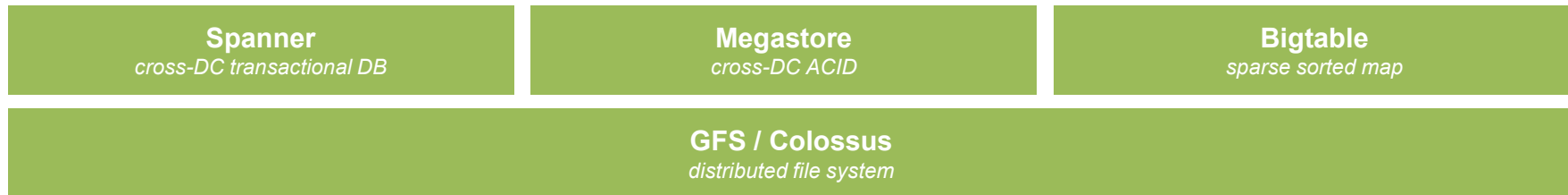
Where RPC sits: one production stack



Data processing



Data storage



Coordination & cluster management



Everything above the bottom row talks to everything else by RPC. It is the narrow waist of the stack.

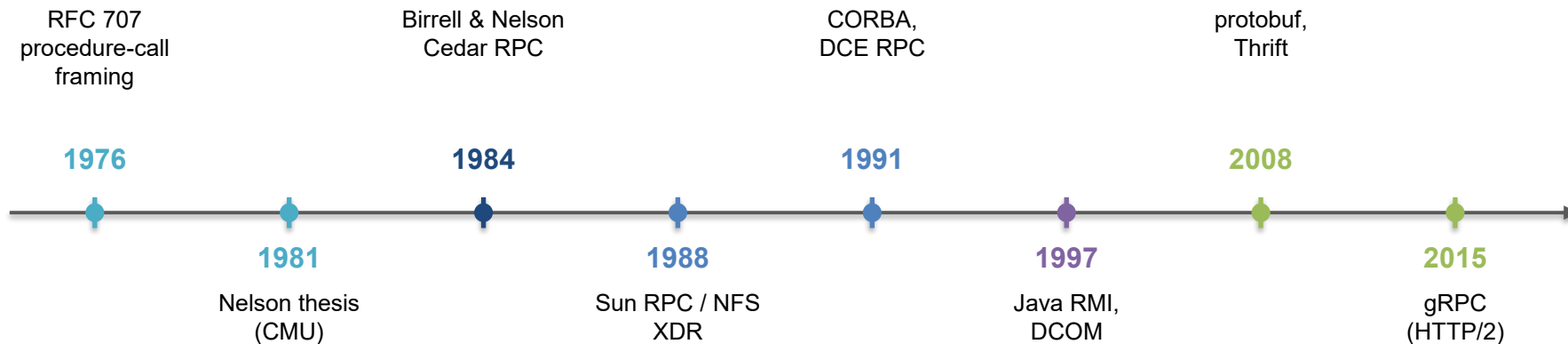
A short history of RPC



COMS4113

Procedure-call-style networking is older than the 1984 paper everyone cites

RPC: five decades



research → infrastructure → objects → web → schema-first binary at scale

The idea is older than the canonical paper; the paper is where it was first made to work well.

Who invented RPC?

- The term and the concept predate the famous 1984 paper.
- **1976** — James E. White, *RFC 707: A High-Level Framework for Network-Based Resource Sharing* proposes a “procedure call” model for ARPANET services.
- **1978** — Per Brinch Hansen, *Distributed Processes: A Concurrent Programming Concept* (CACM 21(11), 934–941) gives a language-level remote procedure construct.
- **1981** — Bruce Jay Nelson’s CMU PhD thesis, *Remote Procedure Call*, is the first systematic treatment and fixes the terminology.
- **1984** — Andrew Birrell & Bruce Nelson, *Implementing Remote Procedure Calls*, ACM TOCS 2(1):39–59 — the Cedar/PARC implementation that made RPC practical and is the standard citation.
- So: **1984 is the definitive implementation paper, not the invention.**

From research to infrastructure

- **Xerox Courier** (1981) — the XNS-era production RPC protocol that grew out of the same PARC work.
- **Sun RPC / ONC RPC** (mid-1980s) — the first mass-deployed RPC; carries NFS. Specified in RFC 1050 (1988), then RFC 1831 (1995), now **RFC 5531** (2009). Uses **XDR** for wire encoding.
- **Apollo NCS** → **DCE RPC** (OSF, early 1990s) — the IDL-plus-stub-compiler model; the direct ancestor of Microsoft **DCOM** and of MSRPC, still load-bearing in Windows today.
- **CORBA** (OMG, 1.0 in 1991) — an ambitious cross-language object broker; declined under specification complexity, uneven interoperability between ORBs, and poor fit with firewalls and the web.
- **Java RMI** (1997, with JDK 1.1) — RPC for a single language, with distributed garbage collection.

The web era, and the return of RPC

- **XML-RPC** (1998) and **SOAP** (1999; W3C Recommendation 2003) put RPC over HTTP and XML — verbose, but firewall-friendly.
- **REST** (Fielding's dissertation, 2000) is *not* RPC: it is a resource-oriented architectural style. It displaced SOAP for public web APIs.
- **Thrift** (Facebook, 2007) brought back the IDL + stub compiler, with many languages and pluggable transports; now Apache Thrift.
- **Protocol Buffers** — Google's internal encoding, open-sourced in **2008**; paired internally with Google's **Stubby** RPC system (never released publicly).
- **gRPC** (2015) is essentially Stubby made public: protobuf over **HTTP/2**, with streaming and deadlines. It is the current default for internal service-to-service RPC.

Why not just use sockets?

The motivation for an abstraction

Sockets are the wrong abstraction

To coordinate, nodes must communicate. The network offers **sockets**: a byte pipe. Everything above a byte pipe is your problem.

```
struct foormsg { uint32_t len; };

void send_foo(int sock, char *contents) {
    int msglen = sizeof(struct foormsg)
                + strlen(contents);
    char *buf = malloc(msglen);
    struct foormsg *fm = (struct foormsg *)buf;
    fm->len = htonl(strlen(contents));
    memcpy(buf + sizeof(struct foormsg),
           contents, strlen(contents));
    write(sock, buf, msglen);
}
```

Sending one string. Now do it for a struct, a list, a map...

What is wrong with this?

Boilerplate

Framing, length prefixes and byte order, hand-written for every message type.

Bugs

This sketch already has three: malloc is unchecked, the return of write is ignored (short writes lose data), and len is sent but never used to bound the read.

Portability

Byte order, alignment, struct padding and word size all differ across machines and compilers.

Evolution

Adding a field means changing every peer at once. There is no versioning story.

The RPC idea

Make network communication look like a **local procedure call**.

Caller (client)

```
z = fn(x, y);  
... do something with z
```

Implementation (server)

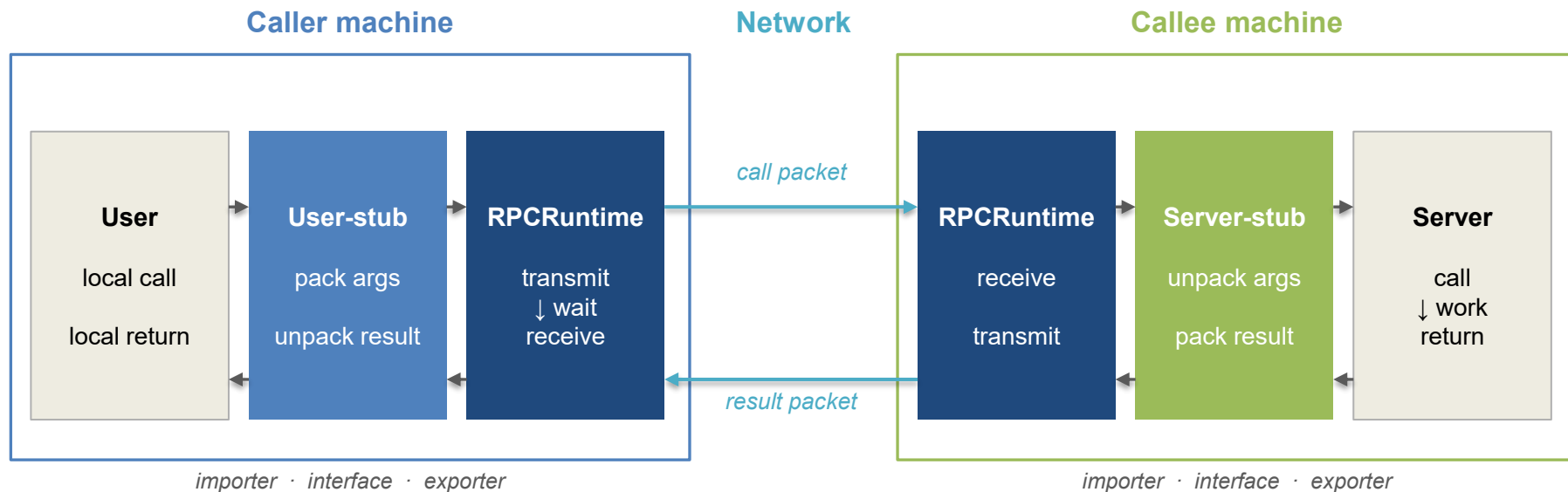
```
fn(x, y) {  
  /* compute z */  
  return z;  
}
```



The two code fragments are unchanged. The library underneath them turns the call into messages, and the messages back into a return value.

The illusion is useful — and it is only an illusion. Latency, pointers and independent failure all leak through.

RPC architecture

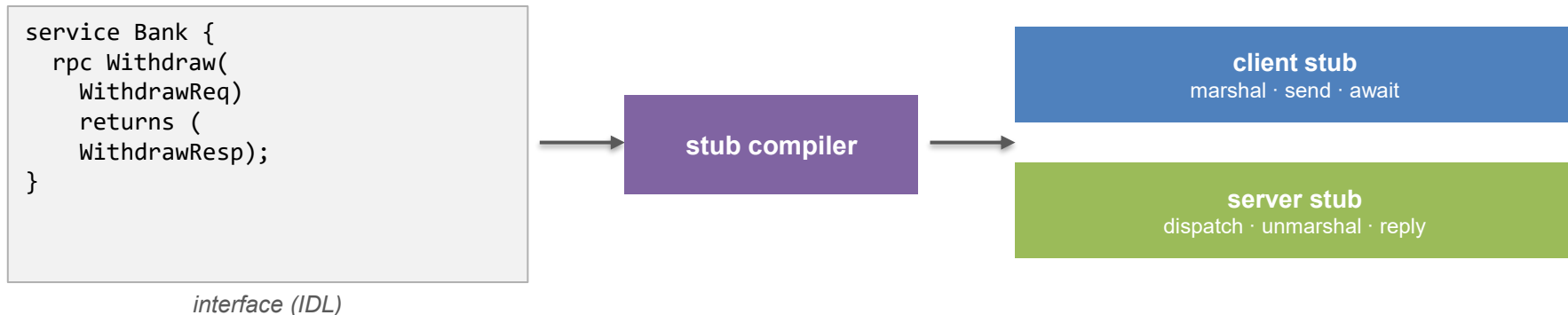


Redrawn from Birrell & Nelson (1984), Figure 1. The user and server code know nothing of packets; the stubs and runtime do all of it.

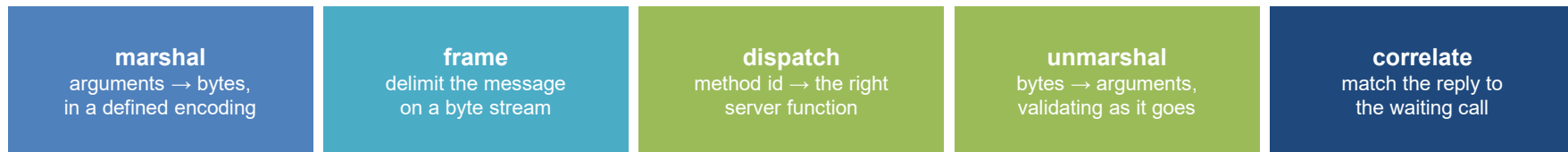
Five hops each way. Any one of them can be where the call is lost — and the caller cannot tell which.

What the stubs actually do

You write an **interface description**; a compiler generates the stubs on both sides.



On every call the stub layer must:



Hand-written socket code must do all five, per message type. This is the boilerplate RPC removes.

- **Familiar.** The call site looks like ordinary code, so every programmer can already read it.
- **Hides encoding.** Marshaling, byte order, alignment and framing are handled once, correctly, in generated code rather than by hand.
- **Independent evolution.** An interface description plus explicit field-compatibility rules lets client and server be rebuilt and redeployed separately.
- **Efficient packing.** A compact binary encoding beats hand-rolled text framing on both size and CPU.
- **Cross-cutting services.** Authentication, authorization, tracing, deadlines, retries, load balancing and metrics attach at the stub layer, uniformly across every call.
- **Location independence.** Naming and discovery decide *where* the call lands, without touching the call site.

... or, where distribution peeks through the local-call illusion

Where the illusion leaks

Latency

A local call is nanoseconds; an RPC is tens of microseconds in a datacenter and tens of milliseconds across the world.

So call granularity is a design decision: batch, pipeline, or go asynchronous. A chatty interface is a slow one.

Pointers & shared memory

A local caller and callee share an address space. A remote pair share nothing.

The library cannot know whether a pointer means “copy this” or “refer to this”, so deep-copy boundaries must be declared. Cyclic structures need explicit handling.

Partial failure

A local call shares fate with its caller: if the callee dies, so do you. Remote components **fail independently**.

A timeout is ambiguous: request lost, server crashed before or after executing, or reply lost. The client cannot distinguish these. *This is the central problem of the course.*

Concurrency & security

Remote calls arrive in parallel, from callers you do not control.

Server state needs real synchronisation, and every argument is untrusted input that must be validated and authorised.

The classic critiques

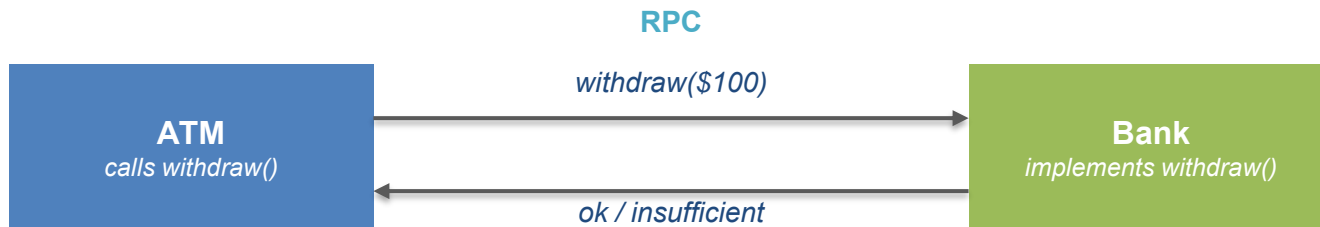


COMS4113

- **Tanenbaum & van Renesse, “A Critique of the Remote Procedure Call Paradigm” (1988).** The synchronous, call-return shape is a poor fit for parallelism, streaming, multicast and heterogeneity.
- **Waldo, Wyant, Wollrath & Kendall, “A Note on Distributed Computing” (Sun Microsystems TR-94-29, 1994).** The influential argument that *unifying* local and remote objects is a mistake: latency, memory access, concurrency and **partial failure** are not implementation details that can be abstracted away.
- Their conclusion is not “don’t use RPC” — it is **don’t let RPC lie to you**. Distribution must be visible in the interface design.
- Related folklore: Deutsch’s “**fallacies of distributed computing**” (Sun, ~1994; later extended to eight) — beginning with “the network is reliable” and “latency is zero”.

Breakout: design an ATM

A customer asks the ATM for \$100. The ATM sends an RPC to the bank; the bank checks the balance, deducts if sufficient, and confirms; the ATM then dispenses the cash.



Now break it. For each failure, what does the customer lose — and who is liable?

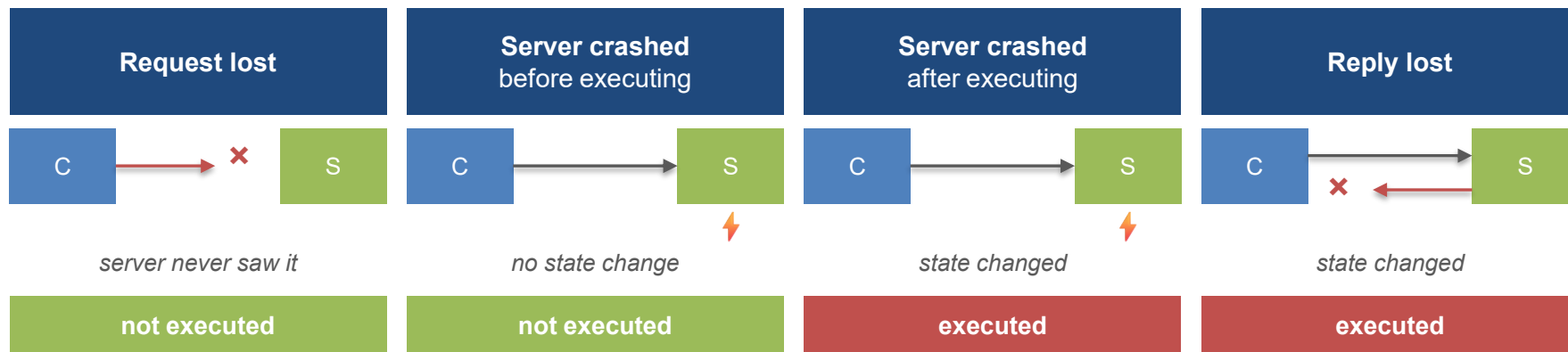
1. The **request** is lost. The ATM retries. Is the customer debited twice?
2. The bank **deducts, then crashes** before replying. The ATM retries. Twice?
3. The **reply** is lost. The money left the account but no cash came out.
4. The bank replies ok, then the **ATM jams** and cannot dispense. Who reverses it?

Every one of these is a *semantics* question, not a coding question. That is the rest of the lecture.

- The client sees one of three outcomes: a reply, an error, or **nothing at all**. The third is the hard one — a timeout does not tell you whether the server executed the call.
- So we must specify what a retry *means*. Three standard semantics:
 - **At least once** — keep retrying until a reply arrives; the call may execute more than once.
 - **At most once** — duplicates are detected and suppressed; the call executes zero or one times.
 - **Exactly once** — the call executes precisely once. The ideal, and the delicate one.
- Note these are *server-side execution* guarantees. What the client can *learn* is weaker: it may never find out which happened.

Why a timeout tells you nothing

The deadline expired. **Four distinct worlds are consistent with that one observation.**



The client's observation is **identical** in all four. It cannot know whether the operation happened.

So the only question the client can actually decide is: **do I retry?** Answering “always” gives at-least-once. Answering “once, and make the server recognise duplicates” gives at-most-once. There is no third option that avoids the choice.

- **Semantic:** the call is eventually executed, possibly more than once.
- **Implementation:** the client retries until it receives a response (ideally with exponential backoff and jitter, plus a deadline).
- If network or server failures are transient, the call eventually succeeds and the semantic holds.
- **When it is safe:** for **idempotent** operations — executing once and executing n times leave the same state.
 - Safe: `read(key)`, `put(key, value)` (absolute write), `delete(key)`.
 - Unsafe: `withdraw($100)`, `append(log, entry)`, `increment(counter)` — all accumulate per execution.
- Non-idempotent operations can often be *made* idempotent by giving the request a unique key and deduplicating on it — which is exactly at-most-once, next.

Semantic: the call executes zero or one times — never twice.

Mechanism: the client tags each *logical* request with a unique id (xid); the server remembers what it has already done.

```
// server, per request
lock(xid)                // serialise duplicates

if (state[xid] == DONE)
    return response[xid]; // replay stored reply

if (state[xid] == IN_PROGRESS)
    return RETRY_LATER;   // do not execute twice

state[xid] = IN_PROGRESS;
x = handler(args);

// <-- a crash here loses the fact that we ran

response[xid] = x;
state[xid] = DONE;
return x;
```

*Retries must reuse the **same** xid. A new id on retry makes this a different logical request, and the guarantee is void.*

Three things to get right

Concurrent duplicates

If a retry arrives while the first call is still running, a bare DONE check lets both execute. An IN_PROGRESS state (or a lock) is required.

Crash between execute and record

The marked line is the hole: the effect is durable but the dedup record is not. On restart the server re-executes. This is exactly what defeats at-most-once, and only an atomic commit of both closes it.

Table growth

state/response grow without bound. Real systems bound them with a client-acknowledged low-water mark, a session, or a TTL — and then the guarantee only holds inside that window.

- **Semantic:** the call executes once, and the client learns that it did.
- This is the ideal — it is what a local procedure call gives you, and it is what programmers assume by default.
- **Exactly-once *delivery* is impossible.** No finite exchange of messages over a lossy link makes both sides *certain* the call ran once — the **Two Generals** problem.
- **Exactly-once *effects* are achievable**, and that is what real systems mean. Either make the operation idempotent, or commit handler(args) **atomically together with** the dedup record state[xid]/response[xid], in durable storage.
- Then “execute” and “remember that we executed” can never come apart: a retry after recovery replays the stored reply instead of re-executing. The call may be *delivered* many times; its *effect* is indistinguishable from once.
- Note what actually defeats it: not packet loss (retry fixes that) but **crash-recovery combined with loss** — the sender can die mid-retry, the server mid-dedup.

The three semantics, side by side

	At least once	At most once	“Exactly once”
Client behaviour	retry until reply	retry with the same xid	retry with the same xid
On a duplicate	re-executes	replays cached reply	replays cached reply
Server state	none needed	dedup table (xid → reply)	dedup table, durable
May execute twice?	yes	no	no
May not execute?	no (if failures are transient)	yes	no (given recovery)
Survives server crash?	n/a	no — the hole above	yes, via atomic commit
Cost	none	memory + GC policy	durable write on the critical path
Use when	the operation is idempotent	duplicates are worse than loss	correctness demands it and you can pay

*“Exactly once” is in quotes on purpose: it is exactly-once **effects** under stated assumptions, not a guarantee the network can give you.*

What you would actually use today

The 2026 landscape

Framework	Encoding	Transport	Use it for
gRPC	protobuf	HTTP/2 (H3 emerging)	internal polyglot services; streaming; deadlines
Connect	protobuf / JSON	HTTP/1.1, /2, /3	browser + backend on one port; gRPC-compatible
Apache Thrift	compact binary	TCP, HTTP	existing Thrift estates; many languages
Cap'n Proto	zero-copy arena	TCP, Unix socket	latency-critical, same-host or trusted links
JSON-RPC / tRPC	JSON	HTTP, WebSocket	small services; TypeScript end-to-end
REST / GraphQL	JSON	HTTP	<i>not RPC</i> — public, third-party-facing APIs
eRPC / RDMA	custom	kernel-bypass NIC	single-digit- μ s datacenter round trips

All of them leave the semantics choice to you. None of them makes partial failure go away.

What changed, and what did not



COMS4113

- **Transport.** HTTP/2 multiplexing removed head-of-line blocking at the application layer; **HTTP/3 over QUIC** removes it at the transport layer too, and is now widely deployed for edge traffic.
- **Browsers.** Browsers cannot speak raw HTTP/2 framing, so **gRPC-Web** and **Connect** exist to bridge them; Connect speaks gRPC, gRPC-Web and its own plain-HTTP protocol on one port.
- **The mesh.** Retries, deadlines, mTLS, load balancing and telemetry increasingly live in a sidecar or proxy (Envoy, Istio, Linkerd) rather than in each client library.
- **Latency floor.** In-datacenter RPC is now bounded by kernel and protocol overhead, not the wire; **kernel-bypass / RDMA** designs such as eRPC reach single-digit-microsecond round trips.
- **What did not change.** Partial failure. Every framework above still makes you choose a retry policy and an idempotency story — exactly the choice Birrell and Nelson faced in 1984.

- Even the most basic distributed-systems abstraction requires substantial machinery to implement with strong semantics.
- “Strong semantics” here means: **behaviour equivalent to the single-machine version of the same program.**
- The obstacle is **independent failure**. A local call shares fate with its caller; a remote call does not, and a timeout is genuinely ambiguous.
- Therefore RPC cannot be fully transparent. Design interfaces so that retries are safe: prefer **idempotent operations with client-supplied request IDs**.
- This same tension — and the same repertoire of fixes (dedup, durable logs, transactions, consensus) — recurs at every layer of the course.

Which library should you use?

- Your homework series includes a deliberately minimal RPC library in the skeleton — enough to expose the semantics problems above.
- For real projects, use a maintained library rather than writing your own. Reasonable defaults in 2026:
 - Internal microservices, polyglot: **gRPC** (protobuf, HTTP/2).
 - Browser-facing or one-port simplicity: **Connect**.
 - Public web API consumed by third parties: **REST/JSON**, or **GraphQL** where clients need to shape their own queries.
 - Extreme in-datacenter latency targets: **RDMA / kernel-bypass** designs; measure before adopting.
- Whichever you pick, the semantics discussion in this lecture is the part you still own.

- Birrell & Nelson, “Implementing Remote Procedure Calls,” **ACM TOCS 2(1):39–59, 1984**.
- Nelson, *Remote Procedure Call*, PhD thesis, Carnegie Mellon, **1981** (CMU-CS-81-119).
- White, *RFC 707: A High-Level Framework for Network-Based Resource Sharing*, **1976**. Brinch Hansen, “Distributed Processes,” **CACM 21(11), 1978**.
- Tanenbaum & van Renesse, “A Critique of the Remote Procedure Call Paradigm,” **EUTECO 1988**.
- Waldo, Wyant, Wollrath & Kendall, “A Note on Distributed Computing,” **Sun Microsystems TR-94-29, 1994**.
- Akkoyunlu, Ekanadham & Huber, “Some Constraints and Tradeoffs in the Design of Network Communications,” **SOSP 1975** (the Two Generals result; named by Gray, **1978**).
- Henning, “The Rise and Fall of CORBA,” **ACM Queue 4(5), 2006**. Srinivasan, **RFC 5531** (Sun RPC v2, 2009).
- Kalia, Kaminsky & Andersen, “Datacenter RPCs can be General and Fast,” **NSDI 2019**.